HTML Tables, Forms, Multimedia & An Introduction to CSS

Introduction to Web Technologies — Week 5

Index

(click to jump — descriptions included at no extra charge)

- 1. Where We Left Off — a two-line recap so nobody feels lost.
- 2. Tables: Rows, Columns, and the Art of `<td>` — spreadsheets before spreadsheets existed.
- 3. Table Headers and the `scope` Attribute — telling screen readers "this column is special."
- 4. Spanning Rows and Columns — Excel's "Merge Cells" button, but you have to type it out.
- 5. Forms: How the Client Finally Talks Back — because browsing isn't just downloading forever.
- 6. Common Form Controls — text boxes, buttons, and everything in between.
- 7. The Query String: Packaging the User's Answers — how "Central Park" becomes `Central+Park`.
- 8. GET vs POST — one puts your password in the address bar; the other doesn't.
- 9. HTML5 Text Input Upgrades — validation without writing a single line of JavaScript.
- 10. Choice Controls: Radio, Checkbox, Select, Datalist — for when typing is too much effort.
- 11. Buttons and Their Many Disguises — submit, reset, image, and the sneaky `<button>`.
- 12. Date and Time Controls — so nobody has to parse "12th of the third, twenty twenty-five" ever again.
- 13. Multimedia: Images, Audio, and Video — HTML grows out of being just text.
- 14. Enter CSS: Separating Content from Looks — HTML builds the house; CSS paints it.
- 15. CSS Syntax: Selectors, Declarations, and Rules — the grammar of styling.
- Practice Questions — five questions to keep you up at night (in a good way).

---

1. Where We Left Off

In the previous session, HTML was introduced as the language that gives the browser a *placeholder* — a structural template — for different kinds of information. HTML says **what** something is; how it actually looks and feels is a job for CSS, which is only briefly previewed here and picked up properly at the end of this session.

This session continues the tour of HTML elements, moving into three new areas: **tables** (structuring data visually), **forms** (letting the client send information back to the server), and **multimedia** (images, audio, and video). It closes with the first look at CSS.

2. Tables: Rows, Columns, and the Art of `<td>`

Whenever information needs to be presented in a grid, HTML gives you the `<table>` element. The interpreter understands a small set of keywords that, together, build the visual grid:

- `<table>` … `</table>` — defines the entire scope of the table.
- `<tr>` … `</tr>` — a **table row**. Each pair of `<tr>` tags is one row.
- `<td>` … `</td>` — **table data**: one cell's worth of content.

A table with two rows and three columns is simply two `<tr>` blocks, each containing three `<td>` blocks:

```
<table>
  <tr>
    <td>15</td>
    <td>15</td>
    <td>30</td>
  </tr>
  <tr>
    <td>45</td>
    <td>60</td>
    <td>90</td>
  </tr>
</table>
```

15	15	30
45	60	90

As soon as a `<td>...</td>` pair appears, the browser treats its content as one column's worth of data for that row. Add more `<td>` pairs to add more columns; add more `<tr>` blocks to add more rows.

> **Note for Review:** `<td>` is *data*; `<th>` (next section) is *header*. They look nearly identical in the markup but mean very different things to a screen reader and to the browser's default styling (headers are bold and centered by default).

3. Table Headers and the `scope` Attribute

Often a table needs labeled rows or columns — this is where `<th>` (**table header**) replaces `<td>` for the labeling cells.

```
<table>
  <tr>
    <th></th>
    <th scope="col">Saturday</th>
    <th scope="col">Sunday</th>
  </tr>
  <tr>
    <th scope="row">Workout Hrs</th>
    <td>6</td>
    <td>4</td>
  </tr>
</table>
```

	Saturday	Sunday
Workout Hrs	6	4

Walking through this: the first row has three cells, but the first one is deliberately left empty (there's no label needed above the row-labels column). The second and third cells are headers with `scope="col"`, meaning "this header describes the column below it."

In the second row, the first cell is a header too — but with `scope="row"`, meaning "this header describes the row to its right." The remaining two cells are ordinary `<td>` data cells.

The `scope` attribute is an attribute–value pair, and it exists specifically to tell assistive technology (and the browser) *which direction* a header applies to — down a column, or across a row.

Beyond structure, tables also support visual styling — cell width, text alignment (centered, left, right justified), and border styling (single, double, or none), along with background colors. These are CSS concerns and are deferred to hands-on exercises and the CSS discussion later in this document.

4. Spanning Rows and Columns

The HTML equivalent of Excel's "Merge Cells" is called **spanning**. Two attributes handle it:

- `colspan="n"` — merges `n` columns *within a row* into a single cell.
- `rowspan="n"` — merges `n` rows *within a column* into a single cell.

Column Spanning

Consider a table listing artworks, where "Size" needs two sub-columns (width and height) but should appear under one combined header:

```
<table>
  <tr>
    <th>Title</th>
    <th>Artist</th>
    <th>Year</th>
    <th colspan="2">Size (width x height)</th>
  </tr>
  <tr>
    <td>The Death of Marat</td>
    <td>Jacques-Louis David</td>
    <td>1793</td>
    <td>162cm</td>
    <td>128cm</td>
  </tr>
  <tr>
    <td>Burial at Ornans</td>
    <td>Gustave Courbet</td>
    <td>1849</td>
    <td>314cm</td>
    <td>663cm</td>
  </tr>
</table>
```

Title	Artist	Year	Size (width x height)	
The Death of Marat	Jacques-Louis David	1793	162cm	128cm
Burial at Ornans	Gustave Courbet	1849	314cm	663cm

The header row now has **only four** `<th>` elements instead of five, because the fourth header's `colspan="2"` makes it visually occupy the space of two columns (the fourth and fifth). Every data row below it still needs its full five `<td>` cells, since the merge only happened at the header level.

Row Spanning

Now flip the problem: one **artist** (Jacques-Louis David) has three paintings listed, and rather than repeating his name three times, the cell is merged vertically using `rowspan`:

```
<table>
  <tr>
    <th>Title</th>
    <th>Artist</th>
    <th>Year</th>
  </tr>
  <tr>
    <td rowspan="3">Jacques-Louis David</td>
    <td>The Death of Marat</td>
    <td>1793</td>
  </tr>
  <tr>
    <td>The Intervention of the Sabine Women</td>
    <td>1799</td>
  </tr>
  <tr>
    <td>Napoleon Crossing the Alps</td>
    <td>1800</td>
  </tr>
</table>
```

Walking through the row count: the first row is the header row (Title, Artist, Year — unchanged). The **second row** starts the data and is the one that actually declares `rowspan="3"` on its Artist cell — this single cell now visually spans across rows 2, 3, and 4.

Because that cell has already claimed the "Artist" column's space for all three rows, the **third and fourth rows** must *omit* the Artist `<td>` entirely — they only contain two cells each (Title and Year), not three. Skipping the rowspan cell in the following rows is not optional; forgetting to omit it is one of the most common table bugs beginners make.

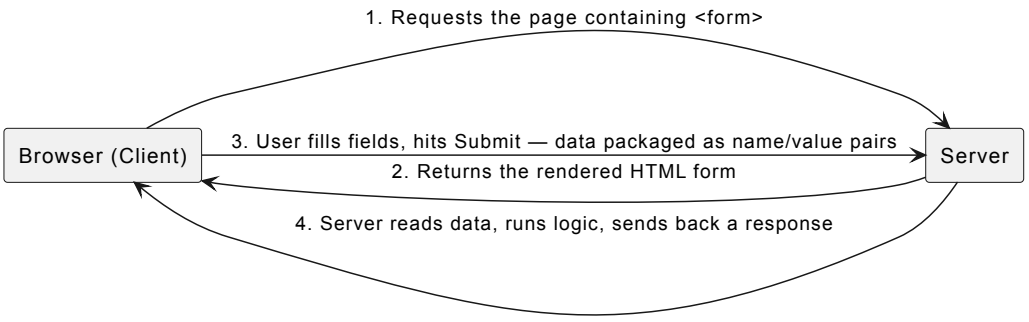
> **Note for Review:** If a genuine fourth painting by a different artist needed to be added afterward, it would go into a brand-new `<tr>` with three ordinary `<td>` cells (Artist, Title, Year) — the rowspan only affects the rows it explicitly covers, not rows added later.

---

5. Forms: How the Client Finally Talks Back

Everything covered so far has been about *displaying* information — a static page renders text, tables, images, or media by fetching content from the server via an HTTP `GET` request. But applications like Google Forms, hotel booking sites, or online quizzes need the reverse flow: the **client sends information to the server**. That is the job of the `<form>` element, and it typically triggers an HTTP `POST` request.

The round trip looks like this:



Step by step: the browser initiates the exchange by requesting a page that contains a `<form>`. The server responds with the rendered form. The user fills in the various fields (text boxes, checkboxes, dropdowns — whatever the form defines) and clicks a submit-type button. That click triggers the packaging of everything the user typed or selected into a set of **name/value pairs**, called a **query string**, which gets sent to the server. The server reads that submitted data, can run whatever logic is needed, and sends a response back — often a redirect to a new page.

6. Common Form Controls

The most familiar form control is the **text/password pair**, seen on virtually every login page:

```
<input type="text" name="username" />
<input type="password" name="password" />
```

Both are single-line entry boxes; `password` additionally masks the typed characters (bullets or similar) for privacy.

Beyond text entry:

- **Radio buttons** force exactly one selection out of a group.
- **Checkboxes** allow several selections at once from the same group.
- **`<select>` + `<option>`** produces a drop-down list.
- **`<textarea>`** provides a free-form, multi-line text box for longer input.
- **`<input type="submit">`** is the button that actually triggers sending the packaged data to the server.

Here is a complete, annotated example of a small form:

```
<form method="post" action="process.php">
  <fieldset>
    <legend>Details</legend>
    <p>
      <label>Title: </label>
      <input type="text" name="title" />
    </p>
    <p>
      <label>Country: </label>
      <select name="where">
        <option>Choose a country</option>
        <option>Canada</option>
        <option>Finland</option>
        <option>United States</option>
      </select>
    </p>
    <input type="submit" />
  </fieldset>
</form>
```

Unpacking this piece by piece:

- `<form method="post" action="process.php">` opens the entire scope of the form. The `method` attribute is either `get` or `post` (covered in Section 8); `action` names the server-side script that will process the submission on success — here, `process.php`. (The server-side technology could just as easily be ASP.NET, plain ASP, or a JavaScript backend; the concept generalizes.)
- `<fieldset>` groups related fields together, and `<legend>` (here, "Details") supplies a heading/label for that group.
- Each `<p>` is simply a structural container to organize a label alongside its input.
- `<label>` is static text shown to the user (e.g., "Title:"); it is **not** the same as the `name` attribute.
- `<input type="text" name="title" />` is the actual editable field. The `name` attribute ("title") is invisible to the end user — it exists purely so the developer (and later, CSS or JavaScript) has a unique handle to refer to this specific field, especially when a form has many text inputs that all need to be styled or processed differently.
- `<select name="where">` behaves the same way: `name="where"` is the developer-facing handle, while the visible `<option>` entries (Choose a country, Canada, Finland, United States) are what the user actually sees and picks from.
- `<input type="submit" />` renders as a clickable button. Clicking it fires the `POST` (or `GET`) request, sending the packaged data to `process.php`.

7. The Query String: Packaging the User's Answers

Once the user submits the form above (typing "Central Park" as the title and choosing "United States" as the country), the browser packages this into a **query string**:

```
title=Central+Park&where=United+States
```

The structure is: `name=value` pairs, each separated by an ampersand (`&`). Here, `title` is the field name and `Central Park` is the value the user typed; `where` is the field name and `United States` is the value the user chose. This pattern holds regardless of whether the input came from a text box, a radio button, or a checkbox — only the *name* of the field and the *value* the user selected or typed matter when building the package.

URL encoding. Not every character can travel safely inside a query string. Spaces are replaced with `+` (as seen above), and other special characters — accented letters, semicolons, and so on — are converted into percent-encoded sequences. For example, an accented "é" becomes `%E9` :

```
artist=Pablo+Jos%E9+Picasso
```

The `%E9` is the URL-encoded form of "é." Any character that isn't safe for a URL gets this treatment so that the query string remains a single, unambiguous line of text.

8. GET vs POST

Both `GET` and `POST` can submit a form, but they package the query string in very different places:

Aspect	GET	POST
Where the query string goes	Appended to the URL	Hidden inside the HTTP request body
Visibility to the user	Fully visible in the address bar	Not visible in the address bar
Browser history / caching	Gets cached, visible in history	Submitted data is not cached
Best suited for	Debugging, non-sensitive lookups, bookmarkable searches	Sensitive data (passwords), large or binary data

Example of what the raw request looks like for each:

```
GET /process.php?title=Central+Park&where=United+States HTTP/1.1
```

```
POST /process.php HTTP/1.1
Date: Sun, 21 May 2017 23:59:59 GMT
Host: www.mysite.com
User-Agent: Mozilla/4.0
Content-Length: 47

title=Central+Park&where=United+States
```

> **Note for Review:** This is a favorite exam distinction. `GET` is convenient for developers debugging a flow (the data is right there in the URL), but it is a poor choice for anything sensitive — passwords sent via `GET` end up sitting in browser history and server logs in plain sight. `POST` hides the payload from the URL and isn't cached, but on its own it is still **not encrypted** — that's what HTTPS (HTTP layered with Transport Layer Security) is for.

9. HTML5 Text Input Upgrades

HTML5 introduced a family of specialized `<input>` types that add lightweight, built-in validation without any custom scripting:

Type	Description
`email`	Single-line box for an email address. Some devices show a specialized keyboard; some browsers validate the format on submit.
`password`	Single-line box; masks entered characters as bullets or similar.
`search`	Single-line box intended for search strings; some browsers add a "clear" icon or distinct styling.
`tel`	Single-line box for telephone numbers. Because formats vary globally, browsers generally don't enforce a specific pattern, though some devices show a numeric keypad.
`text` / `textarea`	Plain single-line (`text`) or multi-line (`textarea`) free text entry. `textarea` also supports a `rows` attribute to control its visible size, and can show placeholder "hint" text that disappears once typing begins.
`url`	Single-line box for a URL; some browsers validate the format on submission.

Two more useful attributes work alongside `type="text"`:

- **`placeholder`** — greyed-out hint text shown inside an empty field (e.g., `placeholder="L#L #L#"` to hint at a postal code format). It vanishes as soon as the user starts typing.
- **`pattern`** — a restriction on which characters/format are acceptable (commonly seen enforcing password complexity — minimum length, at least one capital letter, one special character, etc.). If the entered text doesn't match, the browser shows an error like *"Please match the requested format."*

```
<input type="text" placeholder="L#L #L#" pattern="[a-z][0-9][a-z] [0-9][a-z][0-9]" />
```

10. Choice Controls: Radio, Checkbox, Select, Datalist

Datalist pairs a free-text `<input>` with a suggested list of options — the user can type freely or pick a suggestion:

```
<input type="text" name="city" list="cities" />
<datalist id="cities">
  <option>Calcutta</option>
  <option>Calgary</option>
  <option>London</option>
  <option>Los Angeles</option>
  <option>Paris</option>
  <option>Prague</option>
</datalist>
```

Note the `list="cities"` attribute on the input connects it to the `<datalist id="cities">` block by matching the `id`.

Select can either send the *visible text* as the value, or send a separate numeric/code value using the `value` attribute:

```
<select name="choices">
  <option>First</option>
  <option>Second</option>
  <option>Third</option>
</select>
<!-- If "Second" is picked, query string reads: ?choices=Second -->

<select name="choices">
  <option value="1">First</option>
  <option value="2">Second</option>
  <option value="3">Third</option>
</select>
<!-- If "Second" is picked, query string reads: ?choices=2 -->
```

Radio buttons share the same `name` across the group so that the browser knows only one of them can be selected; `checked` pre-selects a default:

```
<input type="radio" name="where" value="1">North America<br/>
<input type="radio" name="where" value="2" checked>South America<br/>
<input type="radio" name="where" value="3">Asia
```

Checkboxes also share a `name`, but unlike radio buttons, more than one can be checked at once:

```
<label>I accept the software license</label>
<input type="checkbox" name="accept">

<label>Where would you like to visit?</label><br/>
<input type="checkbox" name="visit" value="canada">Canada<br/>
<input type="checkbox" name="visit" value="france">France<br/>
<input type="checkbox" name="visit" value="germany">Germany
```

If the license box and both Canada and Germany are checked, the resulting query string is:

```
?accept=on&visit=canada&visit=germany
```

Notice `visit` appears **twice** — once per checked value — since checkboxes, unlike radio buttons, allow multiple simultaneous selections under the same field name.

---

11. Buttons and Their Many Disguises

Element	Behaviour
`<input type="submit" />`	Sends the form's packaged data to the server.
`<input type="reset" />`	Clears all fields back to their default/empty (null) values — nothing is sent anywhere.
`<input type="button" value="Click Me" />`	A plain clickable button with no built-in form behaviour; typically wired up to custom logic.
`<input type="image" src="appointment.png" />`	Uses an image as a submit-style button.

A `<button>` element can also wrap other HTML — including a hyperlink and an image — giving far more visual flexibility than a plain `<input>`:

```
<button>
  <a href="email.html">
    <img src="images/email.png" alt="..."/>
    Email
  </a>
</button>

<button type="submit">
  <img src="images/edit.png" alt="..."/>
  Edit
</button>
```

The first button uses `<a href="...">` to redirect to another page entirely (an ordinary hyperlink dressed up as a button). The second explicitly sets `type="submit"`, so clicking it behaves like a normal form submission, just with an icon and label inside instead of plain button text.

For quick reference, the broader family of form-related HTML elements:

Type	Description
`<button>`	Defines a clickable button.
`<datalist>`	HTML5 element defining pre-defined value lists to pair with input fields.
`<fieldset>`	Groups related elements in a form together.
`<form>`	Defines the form container.
`<input>`	Defines an input field; HTML5 defines over 20 input types.
`<label>`	Defines a label for a form input element.
`<legend>`	Defines the label for a fieldset group.
`<optgroup>`	Defines a group of related options in a multi-item list.
`<option>`	Defines an option in a multi-item list.
`<output>`	Defines the result of a calculation.
`<select>`	Defines a multi-item list.
`<textarea>`	Defines a multiline text entry box.

---

12. Date and Time Controls

Type	Description
`date`	General date picker. Format: `yyyy-mm-dd`.
`time`	Time picker. Format: `HH:MM:SS` (hours:minutes:seconds).
`datetime`	Combined date and time, including time zone.
`datetime-local`	Combined date and time, **without** a time zone.
`month`	Month-in-a-year picker. Format: `yyyy-mm`.
`week`	Week-in-a-year picker. Format: `yyyy-W##`.

```
<label>Date: </label><br/>
<input type="date" />

<label>Time: </label>
<input type="time" />

<label>DateTime: </label>
<input type="datetime" />

<label>DateTime Local: </label>
<input type="datetime-local" />

<label>Month: </label>
<input type="month" />
```

These controls typically render as native calendar/time pickers in the browser, sparing the developer from writing custom validation for date formats.

13. Multimedia: Images, Audio, and Video

- **`<img src="..." alt="...">`** — loads an image. The `alt` attribute is not just a nice-to-have: it is what a screen reader actually announces, and it's the text shown if the image itself fails to load. In practice, it should never be treated as optional.
- **`<figure>`** wraps an image together with a `<figcaption>`, keeping the caption semantically attached to the image it describes (rather than just being nearby text).
- **`<audio controls>`** and **`<video controls>`** provide native playback controls (play, pause, volume, seek) with no external plugin required.
- Providing multiple `<source>` elements inside `<audio>` / `<video>` lets the browser pick whichever format it actually supports, improving cross-browser compatibility.

HTML5 further extends what can be done with images specifically — controlling color, saturation, and transparency; wrapping text around an image; adding borders; and controlling orientation and position. These are largely CSS-driven capabilities and form a dedicated topic of their own, kept out of scope here beyond this mention.

---

14. Enter CSS: Separating Content from Looks

CSS stands for **Cascading Style Sheets** — a World Wide Web Consortium (W3C) standard for describing how HTML elements should appear.

The core idea: **HTML says what something is; CSS says how it looks.** HTML organizes content into buckets — tables, images, forms, paragraphs, headings — giving the page structure and a template. CSS is layered *on top* of that structure to control appearance: color, spacing, fonts, borders, layout.

CSS can be attached to a page in three ways:

1. **Inline**, directly on an element via the `style` attribute.
2. **Embedded**, inside the `<head>` element of the HTML document.
3. **External**, as a separate `.css` text file linked into the HTML page — the most common approach, since a single stylesheet can style an entire website, which is what makes a codebase maintainable. (Large projects can also split styling across multiple stylesheets — a modular approach — depending on the application's needs.)

This separation of content from presentation is also what makes **responsive design** possible: the same HTML markup can look completely different across devices, purely by changing the CSS applied to it — without touching the underlying structure at all.

15. CSS Syntax: Selectors, Declarations, and Rules

A CSS document is made of one or more **style rules**. Each rule has two parts:

- A **selector**, identifying which HTML element(s) the rule applies to.
- A **declaration block**, wrapped in `{ }`, containing one or more `property: value` pairs separated by semicolons.

```
selector { property: value; property2: value2; }
     |                declaration block               |
                     \_________________/
                       one declaration
```

Two worked examples:

```
em {
  color: red;
}
```

This selects every `<em>` element (recall: `<em>` semantically *emphasizes* text, and browsers render it in italics by default). The declaration sets its `color` property to `red` — so wherever an `<em>` tag appears anywhere in the HTML file, its text turns red.

```
p {
  margin: 5px 0 10px 0;
  font-weight: bold;
  font-family: Arial, Helvetica, sans-serif;
}
```

This selects every `<p>` (paragraph) element. `margin` here follows the top-right-bottom-left convention (5px top, 0 right, 10px bottom, 0 left). `font-weight: bold` makes the text bold, and `font-family` lists a preference order of fonts — Arial first, falling back to Helvetica, and finally to the browser's generic sans-serif font if neither is available.

The **scope** defined by `{ }` is exactly as important here as it is in any programming language (much like block scope in C, C++, or Java) — it precisely delimits which declarations apply to which selected elements. Exactly how far that scope can be pushed — inheritance, specificity, cascading between multiple rules — is picked up in the next session.

Practice Questions

1. Suppose a table's second row uses `rowspan="2"` on its first cell, but the developer forgets to omit that column's `<td>` in the third row. What visual and structural problem would this create, and how would you detect it just by looking at the number of cells per row?

2. A login form is currently built with `method="get"`. List at least two concrete risks this creates, and explain *why* switching to `method="post"` alone still does not make the password fully secure in transit.

3. Two checkboxes and one radio button all share the same `name="option"` attribute in the same form. What would the resulting query string look like if the user checks both checkboxes and selects the radio button — and why does the checkbox behavior differ from the radio button's here?

4. You are asked to build a form field for a product rating where the user should be able to type a custom comment *or* quickly pick from "Excellent," "Good," "Average" without being restricted to only those three. Which HTML5 element combination would you choose, and why is it a better fit than a `<select>`?

5. If all of a website's visual styling is placed inline (via the `style` attribute) on every single element instead of in one external stylesheet, what specific maintenance and responsive-design problems would this create as the site grows to hundreds of pages?